

NORTHERN UNIVERSITY BANGLADESH

LAB REPORT – (Final Submission)

Course Title : Software Development II
Course Code : CSE – 2291
Section : 6(I)
Program : CSE
Project Name : Smart Inventory & Supply Chain Management System

Submitted By:

Rumit Hasan Foysal - 41230301764
Akash Ahmed - 41230301749
Istiak Ahmed - 41230301729
Sadia Farjana Mitu - 41230301727

Submitted To:

Tasfia Tabassum Faija
Lecturer of CSE
Northern University
Bangladesh

Date of Submission: 10-09-2025

Table of Contents

Abstract.....	1
Acknowledgment.....	1
Motivation.....	2
Introduction.....	2
Literature Review	3
Implementation	3
Innovation & Uniqueness	4
Objective.....	5
ER Diagram	6
Appendix A – Table Schemas & Attributes	6
Table 1: users.....	7
Table 2: suppliers.....	7
Table 3: products	7
Table 4: orders	8
Table 5: shipments (outbound).....	8
Table 6: supplier_shipments (inbound).....	8
Table 7: inventory_logs.....	9
Result & Discussion.....	9
Application & Future Scope	10
Conclusion.....	10
References / Bibliography.....	10

Abstract

The **Smart Inventory & Supply Chain Management System** was designed, implemented, and tested to manage products, suppliers, orders, shipments, and auditable stock movements. We engineered a normalized relational schema and implemented business logic using SQL and PL/SQL (views, triggers, stored procedures). The system captures **inbound** (supplier deliveries) and **outbound** (customer order fulfillment) movements in an inventory_logs audit table, with triggers keeping products.stock_quantity accurate. We validated integrity through constraints and realistic sample data, and we demonstrated usage through a lightweight React UI that mirrors database state and showcases query outputs. Using Oracle SQLcl, we exported result sets in **JSON** for modern integration needs. The outcome is a maintainable, scalable, and testable database backbone that reflects real supply chain workflows and is presentation-ready for academic evaluation and future extension.

Acknowledgment

We thank **Tasfia Tabassum Faija**, Lecturer of CSE, for continuous guidance and clear evaluation criteria. We are grateful to department staff and peers for feedback during design reviews and to our lab facility for providing the environment to test Oracle SQL/PLSQL and the React demonstration

Motivation

Modern organizations demand accurate stock visibility, fast order fulfillment, and traceable audit trails for compliance and analytics. Manual spreadsheets and ad-hoc tools create inconsistency, late reorders, and poor supplier coordination. Our motivation was to build a clean, normalized database with enforced integrity and embedded logic that:

1. records every stock movement;
2. separates inbound vs. outbound flows;
3. exposes reliable query able views;
4. Is easy to integrate with a frontend.

Introduction

Problem Statement. Inventory errors, fragmented supplier data, and non-auditable stock adjustments lead to stock outs, overstocking, and delayed shipments.

Scope. We implemented a production-like relational schema with Oracle SQL/PLSQL: users, suppliers, products, orders, outbound shipments, inbound supplier shipments, and an inventory log for auditable movements. Business logic runs inside the database using triggers and stored procedures, and a React UI demonstrates key flows.

Literature Review

Enterprise solutions such as ERP/Inventory suites (e.g., SAP-style modules, Oracle NetSuite-type systems, Odoo/Zoho Inventory-style platforms) provide integrated purchasing, stock, and order management. Academic works emphasize normalization, transaction logging, and trigger-driven consistency for inventory control. Our system follows these principles while tailoring to course goals: a compact, understandable schema, clear separation of **inbound** vs **outbound** logistics, and procedural automation (triggers & procedures) to reduce manual intervention. Compared to generic CRUD tools, our approach prioritizes data integrity, audibility, and query-first design with JSON export to support lightweight UIs.

Implementation

Technologies Used. Oracle Database (XE), SQL, PL/SQL (views, triggers, procedures), Oracle SQLcl (for JSON result export), ER modeling tools (Oracle Data Modeler/SmartDraw), and a React UI to visualize data and simulate operations. Process.

1. Requirements elicitation and entity identification.
2. ERD design; final relational schema with PK/FK and checks.
3. schema.sql to create tables and load sample data.
4. logic_queries.sql with views, triggers, and procedures.
5. Iterative testing with realistic scenarios (order placement, shipment receiving).
6. JSON output verification using SQLcl for demonstration and future app integration.

Business Logic Highlights.

- ✚ View: product + supplier stock view for quick lookups.
- ✚ Triggers: automatic stock updates via inventory logs; outbound shipment record created after an order is inserted.
- ✚ Procedures: place_order (creates order + outbound log); receive_shipment (records inbound supplier delivery + inbound log).
- ✚ Constraints: identity PKs, FK references, status and type checks to enforce correctness.

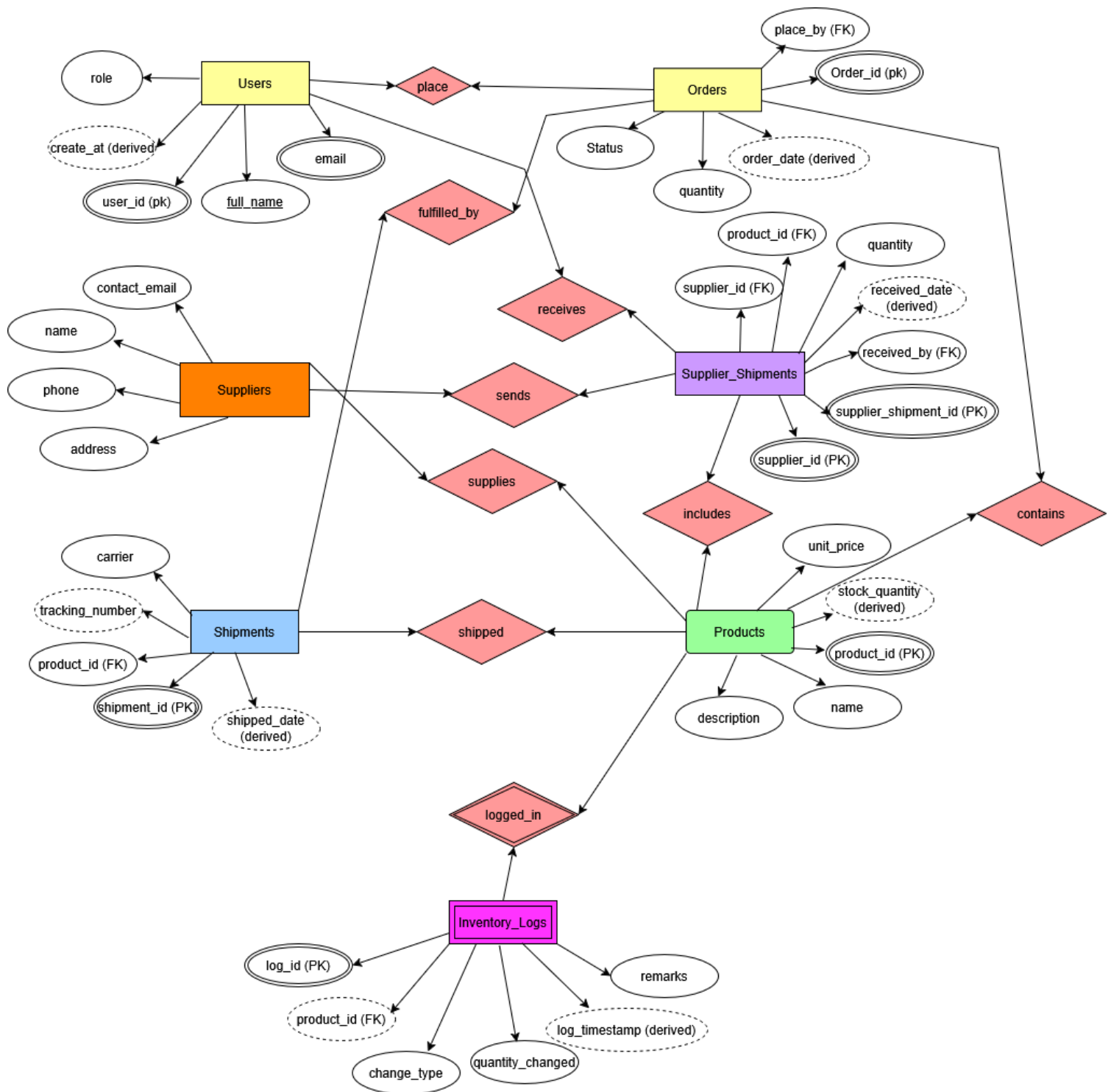
Innovation & Uniqueness

- ✚ **Dual-flow design:** Distinct handling of outbound (customer orders → shipments) and **inbound** (supplier deliveries) reflected in schema and logic.
- ✚ **Auditability by design:** Every stock change is captured in inventory_logs with timestamps and remarks.
- ✚ **Database-centric automation:** Triggers and procedures encapsulate business rules near the data for consistent behavior and simpler clients.
- ✚ **Presentation-ready outputs:** SQLcl **JSON** export enables immediate integration and clean demos.
- ✚ **Professional documentation:** Both conceptual (Chen) and relational ERD are prepared; table schemas consolidated in Appendix A.

Objective

1. Deliver a normalized schema supporting users, suppliers, products, orders, shipments (outbound), supplier shipments (inbound), and inventory logs.
2. Implement integrity constraints (PK, FK, CHECK) and identity keys.
3. Provide procedures and triggers that keep products.stock_quantity correct via audited logs.
4. Expose JOIN-based views/queries that answer operational questions (stock status, supplier sourcing, order fulfillment).
5. Demonstrate outputs in JSON for integration, plus a React UI to visualize and simulate flows.
6. Package artifacts as schema.sql and logic_queries.sql, with testable sample data.

ER Diagram



Appendix A – Table Schemas & Attributes

Table 1: users

- **user_id** NUMBER, identity, PK
- **full_name** VARCHAR2(100)
- **email** VARCHAR2(100), unique, NOT NULL
- **role** VARCHAR2(20), CHECK in {admin, staff}
- **created_at** TIMESTAMP, default CURRENT_TIMESTAMP

Table 2: suppliers

- **supplier_id** NUMBER, identity, PK
- **name** VARCHAR2(100), NOT NULL
- **contact_email** VARCHAR2(100)
- **phone** VARCHAR2(20)
- **address** VARCHAR2(200)

Table 3: products

- **product_id** NUMBER, identity, PK
- **name** VARCHAR2(100), NOT NULL
- **description** VARCHAR2(255)
- **unit_price** NUMBER(10,2)
- **stock_quantity** NUMBER, default 0
- **supplier_id** NUMBER, FK → suppliers.supplier_id

Table 4: orders

- **order_id** NUMBER, identity, PK
- **product_id** NUMBER, FK → products.product_id
- **quantity** NUMBER, NOT NULL
- **order_date** DATE, default SYSDATE
- **status** VARCHAR2(20), CHECK in {pending, shipped, cancelled}
- **placed_by** NUMBER, FK → users.user_id

Table 5: shipments (outbound)

- **shipment_id** NUMBER, identity, PK
- **order_id** NUMBER, FK → orders.order_id
- **shipped_date** DATE
- **carrier** VARCHAR2(100)
- **tracking_number** VARCHAR2(100)
- **product_id** NUMBER, FK → products.product_id

Table 6: supplier_shipments (inbound)

- **supplier_shipment_id** NUMBER, identity, PK
- **supplier_id** NUMBER, FK → suppliers.supplier_id
- **product_id** NUMBER, FK → products.product_id
- **quantity** NUMBER, NOT NULL
- **received_date** TIMESTAMP, default CURRENT_TIMESTAMP
- **received_by** NUMBER, FK → users.user_id

Table 7: inventory_logs

- **log_id** NUMBER, identity, PK
- **product_id** NUMBER, FK → products.product_id
- **change_type** VARCHAR2(20), CHECK in {inbound, outbound}
- **quantity_changed** NUMBER, NOT NULL
- **log_timestamp** TIMESTAMP, default CURRENT_TIMESTAMP
- **remarks** VARCHAR2(100)

Notes: All FK relationships are enforced; identity columns auto-generate keys. Inbound/outbound semantics are centralized in **inventory_logs**; triggers/procedures maintain **products.stock_quantity** accordingly

Result & Discussion

We validated key flows end to end:

- 🔗 Order placement → outbound log → automatic stock decrement occurred consistently via procedure and triggers.
- 🔗 Supplier delivery → inbound log → automatic stock increment updated totals accurately.
- 🔗 JOIN queries confirmed correct relationships (e.g., Products × Suppliers), and a stock view simplified reporting.
- 🔗 SQLcl JSON output confirmed applicability for lightweight frontend integration.

Challenges & Resolutions. We resolved JSON export by switching to SQLcl thin JDBC connection; avoided double stock deduction by ensuring a single source of truth (inventory_logs driven updates); and refined FK relationships to eliminate orphan records.

Application & Future Scope

Applications: retail inventory, warehouse operations, manufacturing WIP tracking, and e commerce back office logistics.

Future Scope: role based dashboards, REST API integration, demand forecasting, barcode/QR intake, supplier SLA analytics, and notification workflows for low stock or delayed shipments.

Conclusion

We delivered a database-first inventory & supply chain system with a robust schema, enforced integrity, and embedded business logic. The design treats inventory_logs as an auditable ledger, separates inbound/outbound logistics, and exposes clean query surfaces. Demonstrations through JSON outputs and a React UI confirm usability and extensibility for future coursework or real deployments.

References / Bibliography

- Oracle Database & PL/SQL Documentation
- Oracle SQLcl User Guide
- Oracle SQL Developer Data Modeler
- Database System Concepts – A. Silberschatz, H. Korth, S. Sudarshan
- Selected ERP/Inventory vendor documentation for feature comparison